

00-2025 风格零基础期末总复习资料

目录

1	编译原理 2025 风格零基础期末突击总复习资料	2
---	--------------------------	---

1 编译原理 2025 风格零基础期末突击总复习资料

1.1 使用方法

这份资料按 Slides/Slides/handout 与三份 Recap PPT 重排复习重点。新老师 PPT 覆盖前端、IR/SSA、中端数据流/符号执行/指针分析和后端代码生成。复习时按下面顺序：

词法分析 -> LL 语法分析 -> IR/SSA -> 数据流分析 -> 符号执行 -> 过程间/指针分析 -> 目标代码/寄存器分配/指令调

旧 Review/compile.pdf 中的 LR、ANTLR、SDD、TAC 不再作为模拟卷主线；其中 TAC/SDD 只在 IR 生成相关 PPT 范围内保留。

1.2 一、Slides 主线题型蓝图

1. 词法分析：正则表达式 -> Thompson NFA -> 子集构造 DFA -> DFA 最小化。
2. LL 语法分析：消除左递归 -> FIRST -> FOLLOW -> 预测分析表 -> LL(1) 判断。
3. IR/SSA：三地址码、基本块、CFG、支配关系、支配边界、phi 插入与 phi 转出。
4. 数据流分析：方向、meet、transfer、gen/kill、worklist、IN/OUT。
5. 符号执行：敏感性定义、CNF、Tseitin、同时可满足证明。
6. 过程间/指针分析：调用上下文、context-sensitive、Andersen 约束、points-to 集合。
7. 后端：目标代码、地址模式、冲突图、寄存器分配、spill、指令调度。

1.3 二、词法分析模板

1.3.1 Thompson NFA

字符 a: start --a--> end
空串 epsilon: start --epsilon--> end
选择 A|B: 新 start epsilon 到 A/B 起点, A/B 终点 epsilon 到新 end
连接 AB: A 终点 epsilon 到 B 起点
闭包 A*: 新 start epsilon 到 A 起点和新 end, A 终点 epsilon 到 A 起点和新 end

答题步骤：

1. 按正则表达式括号从内到外拆分。
2. 每个子表达式套 Thompson 模板。
3. 给 NFA 状态编号。
4. 起点画入箭头，终点画双圈。

1.3.2 子集构造

epsilon-closure(S): 从 S 出发只走 epsilon 边到达的状态集合。
move(S,a): 从 S 出发读 a 到达的状态集合。
Start = epsilon-closure({NFA start}).
DFA 终态: NFA 状态集合中含 NFA 终态。

表格写法:

DFA 状态	NFA 状态集合	on a	on b	终态
A	{...}	B	C	否
B	{...}	B	D	是

1.3.3 DFA 最小化

$P0 = \{\text{终态集合}, \text{非终态集合}\}$

按每个输入字符的转移目标所在组拆分。

重复拆分直到分组不再变化。

每个分组变成一个最小 DFA 状态。

1.4 三、LL 语法分析模板

直接左递归:

$A \rightarrow A \text{ alpha} \mid \text{beta}$

消除:

$A \rightarrow \text{beta } A'$

$A' \rightarrow \text{alpha } A' \mid \text{epsilon}$

FIRST:

$\text{FIRST}(a) = \{a\}$

$\text{FIRST}(\text{epsilon}) = \{\text{epsilon}\}$

$\text{FIRST}(XY)$: 先看 $\text{FIRST}(X)$, 若 X 能推出 epsilon , 再看 $\text{FIRST}(Y)$ 。

FOLLOW:

开始符号 S : $\$$ in $\text{FOLLOW}(S)$

$A \rightarrow \text{alpha } B \text{ beta}$: $\text{FIRST}(\text{beta}) - \{\text{epsilon}\}$ 加入 $\text{FOLLOW}(B)$

$A \rightarrow \text{alpha } B$ 或 beta 能推出 epsilon : $\text{FOLLOW}(A)$ 加入 $\text{FOLLOW}(B)$

预测表:

对 $A \rightarrow \text{alpha}$:

$\text{FIRST}(\text{alpha}) - \{\text{epsilon}\}$ 中每个 a 填 $M[A, a]$

若 $\text{epsilon} \in \text{FIRST}(\text{alpha})$, $\text{FOLLOW}(A)$ 中每个 b 填 $M[A, b]$

表中无多重入口, 则是 $\text{LL}(1)$ 。

2025 原题模板:

```

A -> B C
B -> B P | r
C -> t | u

```

消除左递归:

```

A -> B C
B -> r B'
B' -> P B' | epsilon
C -> t | u

```

FIRST 与 FOLLOW:

```

FIRST(A)={r}
FIRST(B)={r}
FIRST(B')={P,epsilon}
FIRST(C)={t,u}
FOLLOW(A)={$}
FOLLOW(B)={t,u}
FOLLOW(B')={t,u}
FOLLOW(C)={$}

```

预测表:

```

M[A,r] = A -> B C
M[B,r] = B -> r B'
M[B',P] = B' -> P B'
M[B',t] = B' -> epsilon
M[B',u] = B' -> epsilon
M[C,t] = C -> t
M[C,u] = C -> u

```

1.5 四、指令调度模板

目的:

在不改变程序语义的前提下重排指令，减少流水线停顿，提高并行执行效率。

三类依赖:

True dependence / RAW: i 写 x , j 读 x , i 必须在 j 前。
 Anti dependence / WAR: i 读 x , j 写 x , j 不能提前破坏 i 读到的旧值。
 Output dependence / WAW: i 写 x , j 写 x , 二者顺序影响最终写入值。

寄存器重命名:

能消除 WAR 和 WAW。

不能消除 RAW。

答题步骤：

1. 每条指令一个节点。
2. RAW 画实线边。
3. WAR 和 WAW 画虚线边。
4. 对写后写、读后写冲突做寄存器重命名。
5. 对剩余依赖图给一个拓扑序。

1.6 五、支配问题模板

定义：

$Dom(n)$ ：从入口到 n 的每条路径都经过的节点集合。

$PostDom(n)$ ：从 n 到出口的每条路径都经过的节点集合。

$idom(n)$ ： n 的最近严格支配者。

$Dom\ Frontier(n)$ ： n 支配某个前驱，但不严格支配该节点的位置。

计算 Dom ：

$Dom(entry) = \{entry\}$

其他节点 Dom 初值为全集

$Dom(n) = \{n\} \cup \bigcap_{p \in pred(n)} Dom(p)$

迭代到不再变化

SSA 中 ϕ 放置：

变量在多个基本块中被定义时，需要看这些定义块的支配边界。

ϕ 放在多条控制流汇合且不同定义可能到达的位置。

更精确地说：在定义块的迭代支配边界处插入该变量的 ϕ 。

1.7 六、符号执行与 SAT 模板

敏感性：

Path-sensitive：区分不同执行路径。

Flow-sensitive：区分程序点和语句顺序。

Context-sensitive：区分函数调用上下文。

Field-sensitive：区分对象字段。

CNF：

CNF 是子句的合取，每个子句是文字的析取。

$(a \text{ or } \text{not } b) \text{ and } (c \text{ or } d)$

Tseitin:

1. 给每个子表达式引入新变量。
2. 为新变量和子表达式的语义关系写 CNF。
3. 加入根变量为真的子句。
4. 得到与原式同时可满足的 CNF。

常用子句:

$x \leftrightarrow (a \text{ and } b)$
 $(\text{not } x \text{ or } a) \text{ and } (\text{not } x \text{ or } b) \text{ and } (x \text{ or not } a \text{ or not } b)$

$x \leftrightarrow (a \text{ or } b)$
 $(x \text{ or not } a) \text{ and } (x \text{ or not } b) \text{ and } (\text{not } x \text{ or } a \text{ or } b)$

$x \leftrightarrow \text{not } a$
 $(\text{not } x \text{ or not } a) \text{ and } (x \text{ or } a)$

同时可满足证明模板:

$F1 = (a_0 \text{ or } \dots \text{ or } a_n \text{ or } c) \text{ and } (b_0 \text{ or } \dots \text{ or } b_m \text{ or not } c)$
 $F2 = (a_0 \text{ or } \dots \text{ or } a_n \text{ or } b_0 \text{ or } \dots \text{ or } b_m)$

$F1 \rightarrow F2$:

若 $c=\text{false}$, 则第一子句要求某个 $a_i=\text{true}$ 。

若 $c=\text{true}$, 则第二子句要求某个 $b_j=\text{true}$ 。

因此 $F2=\text{true}$ 。

$F2 \rightarrow F1$:

若某个 $a_i=\text{true}$, 令 $c=\text{false}$ 。

若某个 $b_j=\text{true}$, 令 $c=\text{true}$ 。

因此 $F1=\text{true}$ 。

1.8 七、Andersen 指针分析模板

points-to 集合:

$\text{pts}(p) = p$ 能指向的对象集合

四类语句:

$y = \&x: x \text{ in } \text{pts}(y)$
 $y = x: \text{pts}(x) \text{ subset } \text{pts}(y)$
 $*y = x: \text{for } v \text{ in } \text{pts}(y), \text{pts}(x) \text{ subset } \text{pts}(v)$
 $y = *x: \text{for } v \text{ in } \text{pts}(x), \text{pts}(v) \text{ subset } \text{pts}(y)$

闭包步骤:

1. 先处理取地址语句，得到初始 `points-to` 集。
2. 处理复制语句，传播集合包含关系。
3. 处理 `store` 语句，把右侧集合写入左侧指向对象的集合。
4. 处理 `load` 语句，把被指对象的集合读入左侧变量。
5. 有新增元素就继续传播，直到不再变化。

1.9 八、PPT 新增重点

IR/SSA:

三地址码：每条语句最多一个主要运算。
基本块：只有入口第一条和出口最后一条会改变控制流。
CFG：节点是基本块，边是可能的控制流。
SSA：每个变量只有一个定义，汇合处用 `phi` 表示来源选择。

数据流分析：

先判断方向：前向或后向。
再判断性质：`may` 用 `union`，`must` 用 `intersection`。
`transfer` 常写成 `OUT = gen union (IN - kill)`。
`worklist` 到所有 `IN/OUT` 不再变化时停止。

过程间分析：

`context-insensitive` 会合并不同调用点的信息。
`context-sensitive` 会区分调用上下文。
`1-call-string` 用最近一个调用点作为上下文。
精度提高通常会带来更高分析开销。

目标代码与寄存器分配：

目标代码生成要说明 `load`、`store`、`calculation` 和地址模式。
冲突图中，同一程序点同时活跃的变量之间有边。
`k` 个寄存器对应 `k` 着色问题。
如果图无法 `k` 着色，就需要 `spill`。

1.10 九、最后背诵清单

Thompson 五模板
子集构造三操作：`epsilon-closure`、`move`、终态判断
DFA 最小化初分组和拆分
左递归消除公式
FIRST/FOLLOW/预测表规则
RAW/WAR/WAW 定义
寄存器重命名能消除 WAR/WAW
Dom/PostDom/Dom Frontier 定义
支配边界用于 `phi` 放置

数据流方向、meet、transfer、gen/kill

Path/Flow/Context/Field sensitivity

CNF 与 Tseitin 三类子句

Andersen 四类约束

目标代码、冲突图、spill、指令调度